

```

    if (x <= 0.0) return 0.0;
    xu = MAX(0., MIN(mu - 10.*t, x - 5.0*t));
}
sum = 0;
for (j=0; j<18; j++) {
    t = x + (xu-x)*y[j];
    sum += w[j]*exp(a1*(log(t)-lnmu)+b1*(log(1-t)-lnmuc));
}
ans = sum*(xu-x)*exp(a1*lnmu-gammln(a)+b1*lnmuc-gammln(b)+gammln(a+b));
return ans>0.0? 1.0-ans : -ans;
}

Doub invbetai(Doub p, Doub a, Doub b) {
    Inverse of incomplete beta function. Returns  $x$  such that  $I_x(a, b) = p$  for argument  $p$ 
    between 0 and 1.
    const Doub EPS = 1.e-8;
    Doub pp, t, u, err, x, al, h, w, afac, a1=a-1., b1=b-1.;
    Int j;
    if (p <= 0.) return 0.;
    else if (p >= 1.) return 1.;
    else if (a >= 1. && b >= 1.) {
        Set initial guess. See text.
        pp = (p < 0.5)? p : 1. - p;
        t = sqrt(-2.*log(pp));
        x = (2.30753+t*0.27061)/(1.+t*(0.99229+t*0.04481)) - t;
        if (p < 0.5) x = -x;
        al = (SQR(x)-3.)/6.;
        h = 2./(1./(2.*a-1.))+1./(2.*b-1.);
        w = (x*sqrt(al+h)/h)-(1./(2.*b-1)-1./(2.*a-1.))*(al+5./6.-2./(3.*h));
        x = a/(a+b*exp(2.*w));
    } else {
        Doub lna = log(a/(a+b)), lnb = log(b/(a+b));
        t = exp(a*lna)/a;
        u = exp(b*lnb)/b;
        w = t + u;
        if (p < t/w) x = pow(a*w*p, 1./a);
        else x = 1. - pow(b*w*(1.-p), 1./b);
    }
    afac = -gammln(a)-gammln(b)+gammln(a+b);
    for (j=0; j<10; j++) {
        if (x == 0. || x == 1.) return x;
        err = betai(a, b, x) - p;
        t = exp(a1*log(x)+b1*log(1.-x) + afac);
        u = err/t;
        x -= (t = u/(1.-0.5*MIN(1., u*(a1/x - b1/(1.-x)))));
        if (x <= 0.) x = 0.5*(x + t);
        if (x >= 1.) x = 0.5*(x + t + 1.);
        if (fabs(t) < EPS*x && j > 0) break;
    }
    return x;
}

};
const Doub Beta::EPS = numeric_limits<Doub>::epsilon();
const Doub Beta::FPMIN = numeric_limits<Doub>::min()/EPS;

```

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nrl.com/aands>, Chapters 6 and 26.[1]
- Pearson, E., and Johnson, N. 1968, *Tables of the Incomplete Beta Function* (Cambridge, UK: Cambridge University Press).

6.5 Bessel Functions of Integer Order

This section presents practical algorithms for computing various kinds of Bessel functions of integer order. In §6.6 we deal with fractional order. Actually, the more complicated routines for fractional order work fine for integer order too. For integer order, however, the routines in this section are simpler and faster.

For any real ν , the Bessel function $J_\nu(x)$ can be defined by the series representation

$$J_\nu(x) = \left(\frac{1}{2}x\right)^\nu \sum_{k=0}^{\infty} \frac{(-\frac{1}{4}x^2)^k}{k! \Gamma(\nu + k + 1)} \quad (6.5.1)$$

The series converges for all x , but it is not computationally very useful for $x \gg 1$.

For ν *not* an integer, the Bessel function $Y_\nu(x)$ is given by

$$Y_\nu(x) = \frac{J_\nu(x) \cos(\nu\pi) - J_{-\nu}(x)}{\sin(\nu\pi)} \quad (6.5.2)$$

The right-hand side goes to the correct limiting value $Y_n(x)$ as ν goes to some integer n , but this is also not computationally useful.

For arguments $x < \nu$, both Bessel functions look qualitatively like simple power laws, with the asymptotic forms for $0 < x \ll \nu$

$$\begin{aligned} J_\nu(x) &\sim \frac{1}{\Gamma(\nu + 1)} \left(\frac{1}{2}x\right)^\nu & \nu \geq 0 \\ Y_0(x) &\sim \frac{2}{\pi} \ln(x) \\ Y_\nu(x) &\sim -\frac{\Gamma(\nu)}{\pi} \left(\frac{1}{2}x\right)^{-\nu} & \nu > 0 \end{aligned} \quad (6.5.3)$$

For $x > \nu$, both Bessel functions look qualitatively like sine or cosine waves whose amplitude decays as $x^{-1/2}$. The asymptotic forms for $x \gg \nu$ are

$$\begin{aligned} J_\nu(x) &\sim \sqrt{\frac{2}{\pi x}} \cos\left(x - \frac{1}{2}\nu\pi - \frac{1}{4}\pi\right) \\ Y_\nu(x) &\sim \sqrt{\frac{2}{\pi x}} \sin\left(x - \frac{1}{2}\nu\pi - \frac{1}{4}\pi\right) \end{aligned} \quad (6.5.4)$$

In the transition region where $x \sim \nu$, the typical amplitudes of the Bessel functions are on the order

$$\begin{aligned} J_\nu(\nu) &\sim \frac{2^{1/3}}{3^{2/3}\Gamma(\frac{2}{3})} \frac{1}{\nu^{1/3}} \sim \frac{0.4473}{\nu^{1/3}} \\ Y_\nu(\nu) &\sim -\frac{2^{1/3}}{3^{1/6}\Gamma(\frac{2}{3})} \frac{1}{\nu^{1/3}} \sim -\frac{0.7748}{\nu^{1/3}} \end{aligned} \quad (6.5.5)$$

which holds asymptotically for large ν . Figure 6.5.1 plots the first few Bessel functions of each kind.

The Bessel functions satisfy the recurrence relations

$$J_{n+1}(x) = \frac{2n}{x} J_n(x) - J_{n-1}(x) \quad (6.5.6)$$

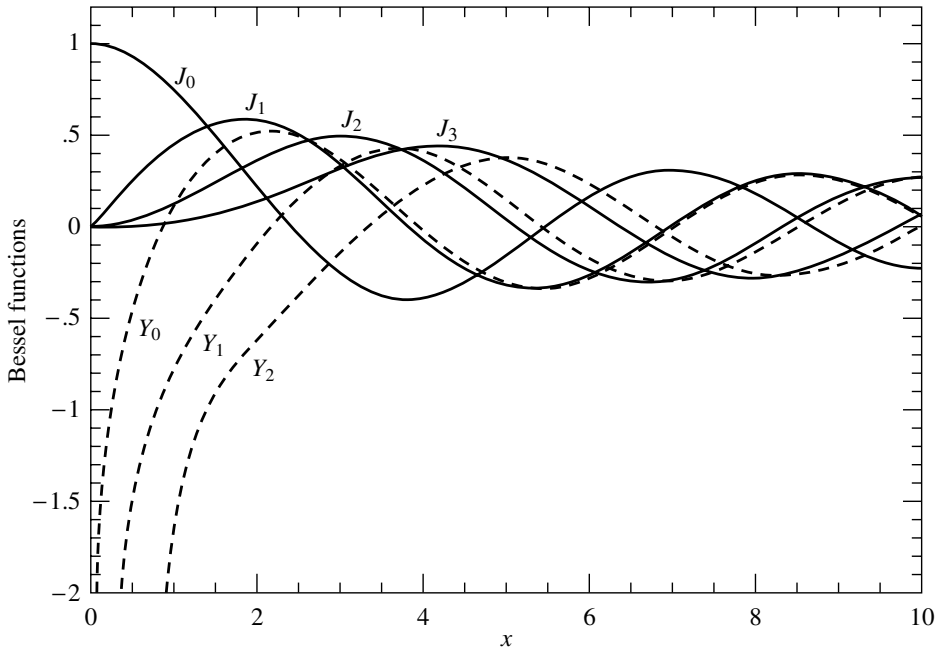


Figure 6.5.1. Bessel functions $J_0(x)$ through $J_3(x)$ and $Y_0(x)$ through $Y_2(x)$.

and

$$Y_{n+1}(x) = \frac{2n}{x}Y_n(x) - Y_{n-1}(x) \quad (6.5.7)$$

As already mentioned in §5.4, only the second of these, (6.5.7), is stable in the direction of increasing n for $x < n$. The reason that (6.5.6) is unstable in the direction of increasing n is simply that it is *the same recurrence* as (6.5.7): A small amount of “polluting” Y_n introduced by roundoff error will quickly come to swamp the desired J_n , according to equation (6.5.3).

A practical strategy for computing the Bessel functions of integer order divides into two tasks: first, how to compute J_0, J_1, Y_0 , and Y_1 ; and second, how to use the recurrence relations stably to find other J ’s and Y ’s. We treat the first task first.

For x between zero and some arbitrary value (we will use the value 8), approximate $J_0(x)$ and $J_1(x)$ by rational functions in x . Likewise approximate by rational functions the “regular part” of $Y_0(x)$ and $Y_1(x)$, defined as

$$Y_0(x) - \frac{2}{\pi}J_0(x)\ln(x) \quad \text{and} \quad Y_1(x) - \frac{2}{\pi}\left[J_1(x)\ln(x) - \frac{1}{x}\right] \quad (6.5.8)$$

For $8 < x < \infty$, use the approximating forms ($n = 0, 1$)

$$J_n(x) = \sqrt{\frac{2}{\pi x}} \left[P_n\left(\frac{8}{x}\right) \cos(X_n) - Q_n\left(\frac{8}{x}\right) \sin(X_n) \right] \quad (6.5.9)$$

$$Y_n(x) = \sqrt{\frac{2}{\pi x}} \left[P_n\left(\frac{8}{x}\right) \sin(X_n) + Q_n\left(\frac{8}{x}\right) \cos(X_n) \right] \quad (6.5.10)$$

where

$$X_n \equiv x - \frac{2n+1}{4}\pi \quad (6.5.11)$$

and where P_0, P_1, Q_0 , and Q_1 are each polynomials in their arguments, for $0 < 8/x < 1$. The P 's are even polynomials, the Q 's odd.

In the routines below, the various coefficients were calculated in multiple precision so as to achieve full double precision in the relative error. (In the neighborhood of the zeros of the functions, it is the absolute error that is double precision.) However, because of roundoff, evaluating the approximations can lead to a loss of up to two significant digits.

One additional twist: The rational approximation for $0 < x < 8$ is actually computed in the form [1]

$$J_0(x) = (x^2 - x_0^2)(x^2 - x_1^2) \frac{r(x^2)}{s(x^2)} \quad (6.5.12)$$

and similarly for J_1, Y_0 and Y_1 . Here x_0 and x_1 are the two zeros of J_0 in the interval, and r and s are polynomials. The polynomial $r(x^2)$ has alternating signs. Writing it in terms of $64 - x^2$ makes all the signs the same and reduces roundoff error. For the approximations (6.5.9) and (6.5.10), our coefficients are similar but not identical to those given by Hart [2].

The functions J_0, J_1, Y_0 , and Y_1 share a lot of code, so we package them as a single object `Bessjy`. The routines for higher J_n and Y_n are also member functions, with implementations discussed below. All the numerical coefficients are declared in `Bessjy` but defined (as a long list of constants) separately; the listing is in a Webnote [3].

```
bessel.h  struct Bessjy {
    static const Doub xj00,xj10,xj01,xj11,twoopi,pio4;
    static const Doub j0r[7],j0s[7],j0pn[5],j0pd[5],j0qn[5],j0qd[5];
    static const Doub j1r[7],j1s[7],j1pn[5],j1pd[5],j1qn[5],j1qd[5];
    static const Doub y0r[9],y0s[9],y0pn[5],y0pd[5],y0qn[5],y0qd[5];
    static const Doub y1r[8],y1s[8],y1pn[5],y1pd[5],y1qn[5],y1qd[5];
    Doub nump,denp,numq,denq,y,z,ax,xx;

    Doub j0(const Doub x) {
    Returns the Bessel function  $J_0(x)$  for any real x.
        if ((ax=abs(x)) < 8.0) {          Direct rational function fit.
            rat(x,j0r,j0s,6);
            return nump*(y-xj00)*(y-xj10)/denp;
        } else {                          Fitting function (6.5.9).
            asp(j0pn,j0pd,j0qn,j0qd,1.);
            return sqrt(twoopi/ax)*(cos(xx)*nump/denp-z*sin(xx)*numq/denq);
        }
    }

    Doub j1(const Doub x) {
    Returns the Bessel function  $J_1(x)$  for any real x.
        if ((ax=abs(x)) < 8.0) {          Direct rational approximation.
            rat(x,j1r,j1s,6);
            return x*nump*(y-xj01)*(y-xj11)/denp;
        } else {                          Fitting function (6.5.9).
            asp(j1pn,j1pd,j1qn,j1qd,3.);
            Doub ans=sqrt(twoopi/ax)*(cos(xx)*nump/denp-z*sin(xx)*numq/denq);
            return x > 0.0 ? ans : -ans;
        }
    }
}
```

```

Doub y0(const Doub x) {
Returns the Bessel function  $Y_0(x)$  for positive x.
    if (x < 8.0) {                                Rational function approximation of (6.5.8).
        Doub j0x = j0(x);
        rat(x,y0r,y0s,8);
        return nump/denp+twoopi*j0x*log(x);
    } else {                                       Fitting function (6.5.10).
        ax=x;
        asp(y0pn,y0pd,y0qn,y0qd,1.);
        return sqrt(twoopi/x)*(sin(xx)*nump/denp+z*cos(xx)*numq/denq);
    }
}

Doub y1(const Doub x) {
Returns the Bessel function  $Y_1(x)$  for positive x.
    if (x < 8.0) {                                Rational function approximation of (6.5.8).
        Doub j1x = j1(x);
        rat(x,y1r,y1s,7);
        return x*nump/denp+twoopi*(j1x*log(x)-1.0/x);
    } else {                                       Fitting function (6.5.10).
        ax=x;
        asp(y1pn,y1pd,y1qn,y1qd,3.);
        return sqrt(twoopi/x)*(sin(xx)*nump/denp+z*cos(xx)*numq/denq);
    }
}

Doub jn(const Int n, const Doub x);
Returns the Bessel function  $J_n(x)$  for any real x and integer  $n \geq 0$ .

Doub yn(const Int n, const Doub x);
Returns the Bessel function  $Y_n(x)$  for any positive x and integer  $n \geq 0$ .

void rat(const Doub x, const Doub *r, const Doub *s, const Int n) {
Common code: Evaluates rational approximation.
    y = x*x;
    z=64.0-y;
    nump=r[n];
    denp=s[n];
    for (Int i=n-1;i>=0;i--) {
        nump=nump*z+r[i];
        denp=denp*y+s[i];
    }
}

void asp(const Doub *pn, const Doub *pd, const Doub *qn, const Doub *qd,
Common code: Evaluates asymptotic approximation.
    const Doub fac) {
    z=8.0/ax;
    y=z*z;
    xx=ax-fac*pio4;
    nump=pn[4];
    denp=pd[4];
    numq=qn[4];
    denq=qd[4];
    for (Int i=3;i>=0;i--) {
        nump=nump*y+pn[i];
        denp=denp*y+pd[i];
        numq=numq*y+qn[i];
        denq=denq*y+qd[i];
    }
}
};

```